JavaScript Notes

Type Conversion • Dates & Times • Functions — with working examples

---

1. Converting Strings and Numbers

String -> Number (arithmetic operators)

When a string that looks like a number appears with **-**, *****, or **/**, JavaScript automatically converts it to a number to do the math.

```
var currentAge = prompt("Enter your age.");   // string, e.g. "32"
var yearsEligibleToVote = currentAge - 18;    // auto-converted to number

var profit = "200" - "150";   // profit = 50 (number)
var profit = "200" - "duck";  // profit = NaN (Not a Number)
```

The + trap: strings win over numbers

With the **+** operator, JavaScript does the opposite: it converts numbers to strings and concatenates instead of adding.

```
var result = "200" + 150;   // "200150"  (concatenation, not addition)

var currentAge = prompt("Enter your age.");
var qualifyingAge = currentAge + 1;
// if user enters 52 -> qualifyingAge = "521"  (bug!)
```

Fix: convert the string to a number first with parseInt.

```
var qualifyingAge = parseInt(currentAge) + 1;   // now does real math
```

parseInt vs parseFloat vs Number

Method	Converts to	Decimal handling	Example
parseInt()	Integer	Chops off decimals (no rounding)	parseInt("1.9999") -> 1
parseFloat()	Floating-point	Keeps decimals	parseFloat("1.9999") -> 1.9999
Number()	Integer or float	Matches the string exactly	Number("24.9876") -> 24.9876

From your app.js — try it yourself:

```
var a = "23.56";
console.log(parseInt(a));      // 23

var a = "23.56";
console.log(parseFloat(a));    // 23.56

var a = "23";
```

```
console.log(Number(a));        // 23

var a = "23";
console.log(+a);                // 23  (unary + also converts to number)

var age = prompt("enter your age");
console.log(age - 2);           // subtraction forces numeric conversion
```

Number -> String

```
var numberAsNumber = 1234;
var numberAsString = numberAsNumber.toString();   // "1234"

// from app.js:
var num = 22;
console.log(num.toString());
```

Controlling decimal length: toFixed()

```
var total = price + (price * taxRate);   // e.g. 10.59675
var prettyTotal = total.toFixed(2);      // "10.60"
var currencyTotal = "$" + prettyTotal;   // "$10.60"
var prettyTotal = total.toFixed();       // no decimals at all

// from app.js:
var num = 4.454442;
console.log(num.toFixed(2));   // "4.45"
```

Note: Browser quirk: toFixed() doesn't always round .5 consistently — sometimes rounds down instead of up. A manual fix bumps a trailing 5 up to 6 before rounding, to force correct rounding.

2. Working with Dates and Times

Creating a Date object

```
var rightNow = new Date();            // current date & time
var dateString = rightNow.toString(); // convert Date to string if needed
```

A Date object is **not** a string — you can't use string methods like charAt or slice on it directly. The date/time comes from the user's own computer clock, so it's only as accurate as their system settings.

Extracting parts of a date

Method	Gets	Range	Example
getDay()	Day of week	0-6	0 is Sunday
getMonth()	Month	0-11	0 is January
getDate()	Day of month	1-31	1 is 1st of month
getFullYear()	Year	—	2015
getHours()	Hour	0-23	0 = midnight, 12 = noon
getMinutes()	Minute	0-59	—
getSeconds()	Second	0-59	—
getMilliseconds()	Millisecond	0-999	—
getTime()	Ms since Jan 1, 1970	—	—

Example: getting the day / month name

```
var dayNames = ["Sun","Mon","Tue","Wed","Thu","Fri","Sat"];
var now = new Date();
var theDay = now.getDay();          // e.g. 3
var nameOfToday = dayNames[theDay]; // "Wed"

// from app.js:
var a = new Date();
var day = ["Sun","Mon","Tues","Wed","Thu","Fri","Sat"];
console.log(day[a.getDay()]);

var c = new Date();
var month = ["January","February","March","April","May","June",
             "July","August","September","October","November","December"];
console.log(month[c.getMonth()]);
```

Specifying a future/past date

```
var doomsday = new Date("June 30, 2035");
// month spelled out, comma after day, 4-digit year
```

```
var d = new Date("July 21, 1983 13:25:00");
// time: no comma, 24-hour, colons between h:m:s
```

Example: counting time between two dates

```
var msDiff = new Date("June 30, 2035").getTime() - new Date().getTime();
var daysTillDoom = Math.floor(msDiff / (1000 * 60 * 60 * 24));
// ms -> seconds -> minutes -> hours -> days, rounded down

// from app.js (years until the 2030 World Cup):
var fifaWorldCup = new Date("25 June 2030");
var nextfifaWorldCup = fifaWorldCup.getTime();
var currentTime = new Date();
var c_m_s = currentTime.getTime();
console.log(((((nextfifaWorldCup - c_m_s) / 1000) / 60) / 60) / 24 / 365);
```

Changing parts of a date (set methods)

Method	Example	Result
setFullYear()	d.setFullYear(2006);	Year becomes 2006
setMonth()	d.setMonth(6);	Month becomes 6 (July)
setDate()	d.setDate(6);	Day of month becomes 6
setHours()	d.setHours(6);	6 a.m.
setMinutes()	d.setMinutes(6);	6 minutes past the hour
setSeconds()	d.setSeconds(6);	6 seconds past the minute
setMilliseconds()	d.setMilliseconds(6);	6 ms past the second

Each set method changes only that one element — every other part of the date/time stays the same.

Putting it together: a clock function (from app.js)

```
function tellTime() {
    var a = new Date();
    alert("Current time" + " " + a.getHours() + ":" + a.getMinutes() + ":" + a.getSe
conds());
}
```

3. Functions

Why use a function?

A function packages a block of code under one name, so instead of repeating the code everywhere it's needed, you just call the function's name.

```
function tellTime() {
    var now = new Date();
    var theHr = now.getHours();
    var theMin = now.getMinutes();
    alert("Current time: " + theHr + ":" + theMin);
}
tellTime();   // this one line runs the whole block
```

Passing data in: arguments and parameters

```
function greetUser(greeting) {
    alert(greeting);
}
greetUser("Hello, there.");
// "Hello, there." is the argument
// greeting is the parameter that catches it
```

The argument's name and the parameter's name don't need to match — JavaScript matches values to parameters by their **order**, not by name.

```
function showMessage(m, string, num) {
    alert(m + string + num);
}
var month = "March";
showMessage(month, "'s winner number is ", 23);
// alert: "March's winner number is 23"

// from app.js:
function greet(name) {
    console.log(name + " Hello");
}
greet("Faizan");   // "Faizan Hello"
```

Passing data back: return

A function can also send a value back to whatever code called it. This is done with the **return** keyword, followed by the value (or variable holding the value) you want to send back.

```
function calcTot(merchTot) {
    var orderTot;
    if (merchTot >= 100) {
        orderTot = merchTot;
    } else if (merchTot < 50.01) {
        orderTot = merchTot + 5;
    } else {
        orderTot = merchTot + 5 + (.03 * (merchTot - 50));
```

```
    }
    return orderTot;
}
var totalToCharge = calcTot(79.99);   // catches the returned value
```

Shipping rule used above: $5 minimum, plus 3% of anything over $50, free at $100+.

```
// from app.js:
function add_2(a, b) {
    return (a + b);
}
a = add_2(2, 6);   // 8

if (a % 2 == 0) {
    console.log("even");
} else {
    console.log("Odd");
}
// -> "even"
```

How the two-way communication works

- Calling code passes a value in -> calcTot(79.99)
- The parameter (merchTot) catches it inside the function
- The function processes it and stores the result in a local variable (orderTot)
- return orderTot; sends that value back out of the function
- The calling code needs a variable to catch it, e.g. totalToCharge

The statement `var totalToCharge = calcTot(79.99);` is really shorthand for: "Assign to totalToCharge the value returned by the function calcTot." The variable that returns the value inside the function and the variable that catches it outside don't need to share a name — they're two separate variables in two separate scopes.

A function used anywhere a variable can be used

Because a function call resolves to whatever it returns, you can drop it in wherever you'd normally use a literal value, a variable, or an expression:

```
// Directly in an alert
alert(calcTot(79.99));

// Inside an arithmetic expression
var orderTot = merchTot + calcTax(merchTot);

// As an argument passed to another function
var tot = calc(merchTot, calcTax(merchTot));

// Called from inside another function
function calcTot(price) {
```

```
    return price + calcShip(price);   // calcTot calls calcShip
}
```

In the last example, calcTot calls calcShip, passes it price, and immediately adds the returned shipping cost to price before returning its own total — one function's return value feeding directly into another calculation.

Only ONE value can come back

A function can accept many parameters, but no matter how much processing it does, it can only return a **single value** to the code that called it. If several pieces of data need to come back, bundle them into one array or one object and return that single array/object.

A function with no return

Not every function needs to return something. Functions like tellTime() or greetUser() just perform an action (like showing an alert) and don't send any value back — that's perfectly valid. Use return only when the calling code needs to use a result afterward.

Key rule about return

A function can take any number of parameters, but it can only return one single value back to the code that called it.